Hoja de sintaxis de Python

Tecnología e Ingeniería I · 1.º de Bachillerato

Variables y tipos

```
intensidad = 3        # int: entero
tension = 12.5        # float: decimal (con punto)
unidad = "V"          # str: texto, entre comillas
encendido = True      # bool: True o False
print(type(tension))  # <class 'float'>
```

= guarda («calcula lo de la derecha y guárdalo»); == compara. Nombres sin espacios, ni empezar por número: `velocidad_kmh`.

Entrada, salida y conversión

```
nombre = input("Nombre: ")       # str
v = float(input("Velocidad: "))  # número
print("Hola,", nombre)    # Hola, Ada
print("A", "B", sep="-")  # A-B
print(n, end=" ")         # no salta línea
print(round(27.777, 2))   # 27.78
```

`int("12")` → 12 · `float("3.5")` → 3.5 · `str(7)` → `"7"`. Con textos, + pega: `"50" + "5"` → `"505"`.

Operadores

- `+` `-` `*` suma, resta y producto: `7 * 2` → 14
- `/` división, siempre da float: `7 / 2` → 3.5
- `//` división entera: `7 // 2` → 3
- `%` resto de la división entera: `7 % 2` → 1
- `**` potencia: `2 ** 10` → 1024

Orden de las matemáticas; los paréntesis lo cambian.

Comparación: `==` `!=` `<` `>` `<=` `>=` → `True` o `False`. Lógicos: `and` (las dos), `or` (basta una), `not` (da la vuelta).

Condicionales

```
if I <= 10:
    print("Consumo bajo")
elif I <= 16:
    print("Consumo medio")
else:
    print("Consumo alto")
```

Se prueba de arriba abajo y solo se ejecuta la primera que se cumple. Línea con : y lo que depende, sangrado 4 espacios.

Bucle while y centinela

```
total = 0          # acumulador, antes del bucle
n = 0              # contador, antes del bucle
dato = input("Potencia (o fin): ")
while dato != "fin":
    total = total + float(dato)
    n = n + 1
    dato = input("Potencia (o fin): ")
print(n, total)
```

Dentro del bucle algo tiene que cambiar la condición; si no, es infinito (Stop).

Bucle for y range

```
for i in range(5):     # i vale 0, 1, 2, 3, 4
    print(i)
```

- `range(2, 7)` → 2, 3, 4, 5, 6
- `range(0, 10, 3)` → 0, 3, 6, 9

El fin **nunca** se incluye; solo enteros. Paso decimal: `for i in range(11):` y `t = i * 0.5`.

Listas

```
calibres = [6, 10, 16, 20]
calibres[0]          # 6 (las posiciones empiezan en 0)
calibres[-1]         # 20 (el último)
len(calibres)        # 4
calibres.append(25)  # añade al final
for c in calibres:                  # los datos
    print(c)
for i in range(len(calibres)):      # las posiciones
    print(i, calibres[i])
```

El mayor y su posición (el menor, igual con <):

```
mayor = medidas[0]
pos = 0
for i in range(len(medidas)):
    if medidas[i] > mayor:
        mayor = medidas[i]
        pos = i
```

Buscar con bandera:

```
encontrado = False
for c in calibres:
    if c == buscado:
        encontrado = True
```

Funciones

```
def Rserie(R1, R2):
    """Resistencia equivalente en serie."""
    return R1 + R2

Req = Rserie(100, 220)   # los datos van a R1 y R2
```

`return` devuelve el resultado y termina la función; sin `return`, la función devuelve `None`. Las variables creadas dentro no existen fuera.

Errores

```
try:
    v = float(texto)
except ValueError:
    print("Eso no es un número")
```

- `SyntaxError`: Python no entiende lo escrito (falta :, un paréntesis, unas comillas).
- `NameError`: un nombre que no existe (mal escrito o local).
- `ValueError`: un texto que no se puede convertir en número.
- `IndexError`: una posición que no existe en la lista.
- De lógica: no avisa, pero el resultado está mal. Prueba con un caso conocido y traza.